DATA ENGINEERING

ASSIGNMENT - 03

A. AKSHAYA
CSD
2UNU1AU404

Q1. a) Define data architecture. Explain how choosing common components wisely and building loosely coupled systems lead to better data architecture.

Ans: Data architecture is the design or blueprint that defines how data is collected, stored, processed and accessed within an organization so that it can be used efficiently & securely.

*CHOOSING COMMON COMPONENTS WISELY :-

- Using common components helps improve collaboration and knowledge sharing among teams.
- It also avoids reinventing tools, improving efficiency and standardization.
- Data engineers should choose common components that can be used by multiple teams across the organization.

*BUILDING LOOSELY COUPLED SYSTEMS :-

- Here, systems are divided into small independent components.
- The components communicate through API's / service interfaces.
- changes in one component do not affect the other's which allows teams to develop, test, and update systems independently.

b) Discuss how a data engineer can apply microservices principles to a traditionally monolithic central data warehouse.

Ans :- APPLYING MICROSERVICES PRINCIPLES TO A MONOLITHIC CENTRAL DATA WAREHOUSE

A monolithic data warehouse stores and processes all organizational data in a single centralized system, which leads to tight coupling and high complexity. A data engineer can apply microservices principles to make the architecture more flexible & scalable

1. Break the monolithic system into smaller services :

- Instead of one large warehouse handling everything, divide the system into smaller domain-specific components.

2. Use domain-specific data pipelines :

- Create separate pipelines for different domains.
- Each pipeline processes and manages data related only to its domain.

3. Connect pipelines to domain-specific data warehouse :

- For example, the sales pipeline connects to a sales data warehouse, while inventory data goes to an inventory warehouse.

4. Reduce tight coupling :

- Each service operates independently, so failure or change in one service does not affect other services.

5. Enable scalability and flexibility :

- Independent services allow teams to develop, update and scale systems separately, improving efficiency & performan

Illustrate with an example how atomicity and consistency of ACID protect against transaction failures in an online payment system.

Ans: ACID PROPERTIES :

ACID is a set of database characteristics that ensure reliable transactions.

ACID stands for :
- Atomicity
- Consistency
- Isolation
- Durability

1. Atomicity -

→ Means a transaction is executed completely or not executed at all.

- If any step fails, the entire transaction is rolled back.
- This prevents partial updates in the database.

Eg - Online Payment

If a user transfers ₹500 from Account A → B :

- Step 1 : Deduct ₹500 from Account A
- Step 2 : Add ₹500 to Account B

If step 2 fails due to system error, step 1 is cancelled & money is returned to Account A.

Thus, either both steps happen or none happen.

2. Consistency -

→ This ensures that the database always remains in a valid and correct state after a transaction.

- Any database read returns the last written version of data.

Eg :-

Before transaction :

- Account A = ₹ 2000
- Account B = ₹ 1000

After transaction of ₹ 500 :

- Account A = ₹ 1500
- Account B = ₹ 1500

The database reflects correct & updated balances, maintaining consistency.

Q3. Explain briefly about batch, continuous CDC and database replication.

Ans:- 1. Batch-Oriented CDC :-

Batch CDC captures database changes periodically in batches.

- Uses fields like updated_at timestamp to identify changed rows.
- Retrieves rows updated since the last capture time.
- Limitation : only the latest updated record is captured, intermediate changes may be missed.

2. Continuous CDC :-

captures every change in the database in near real-time.

- Treats each database write as an event.
- uses database logs to record changes sequentially.
- Useful for real-time data ingestion & streaming analytics.

CDC & Database Replication :-

CDC can also be used for replicating data between databases.

- Synchronous replication - replica stays fully synchronized with the primary database.
- Asynchronous CDC replication allows loosely coupled architecture & may have slight delay in updates.